SPE UMaT Chapter Website — Developer Documentation

How to Use This Document

This document turns the technical committee's proposal for the SPE UMaT Chapter website into two self-contained developer briefs, so frontend and backend work can be assigned and progressed independently.

- **Part A — Frontend Development:** the public website, the member-facing registration, merit lookup and check-in pages, and the admin section (React + TypeScript).
- **Part B — Backend & Infrastructure Development:** the database, Row Level Security, Edge Functions, membership ID generation and the check-in logic (Supabase).
- **Appendix:** the shared database schema, architecture, a proposed timeline, the access the committee still needs to provide, and key risks.

Contract between the two parts

- Visitors and members have no accounts. Their three actions (register, look up merit, check in) go through Edge Function endpoints only. The frontend never reads the members table directly.
- Only admins sign in, through Supabase Auth. Admin screens read data under RLS.
- The backend owns every secret (service role key, CAPTCHA secret). The frontend only holds public/anon keys.
- The schema in Appendix 1 is the single source of truth. Any change is agreed by both developers before merging.

conventions

#	Item	Frontend	Backend
1	Homepage	A.2	B.2
2	About page	A.2	B.2
3	Website research and inspiration	A.2	—
4	Member registration and 6-digit membership ID	A.2, A.3, A.4	B.3, B.4
5	Merit lookup by membership	A.2, A.3	B.3, B.6
6	Administrative section and registrations export	A.4, A.6	B.2, B.8
7	News, media and content management	A.2, A.4	B.2
8	Merit Tracker and event QR check-in	A.3, A.4	B.5, B.6

Part A — Frontend Development

Scope: the React + TypeScript application covering the public chapter website, the member-facing pages (registration, merit lookup, event check-in) and the executive admin section, in one codebase and one deployment. Members have no login and no accounts.

A.1 Tech Stack

Layer	Choice
Framework	React 18+ with TypeScript (Vite)
Styling / UI	Tailwind CSS (Proposed), themed to SPE / UMaT chapter branding
State management	React Context API or Zustand
Backend client	supabase-js with the anon key (admin screens and public content); fetch calls to Edge Functions for member actions
Forms & validation	React Hook Form + Zod
Charts	Recharts (Proposed) for the admin overview
QR codes	qrcode.react to generate the live, auto-refreshing event QR in the admin portal. Members scan with their phone's own camera, so no scanner library is needed
Export	Client-side CSV and Excel export of the registrations table (Proposed: papaparse and SheetJS)
Hosting	Vercel

A.2 Public Pages

None of these routes need a login.

Route	Purpose	Contents
/	Homepage	Brief introduction to the Society of Petroleum Engineers (SPE); overview of the SPE UMaT Chapter; pictures of events, activities, members and executives; latest news; upcoming events with links to event details; Contact Us section
/about	About	SPE mission and vision; information on the SPE UMaT Chapter; benefits of SPE membership; other information for prospective members
/events, /events/:slug	Events	Upcoming and past events, each with a detail page and cover image

Route	Purpose	Contents
/news, /news/:slug	News	News and announcements, newest first, each with a date, cover image and detail page
/media	Media	YouTube videos and links, event calendar, workshop recordings, picture galleries
/register	Registration	Form with first name, last name, programme, year, email and phone number. Emails a verification code through Resend before any ID is issued
/register/verify, /register/success	Email verification and result	Member enters the verification code from their email. On success the page shows the new 6-digit ID clearly, with a copy button and a reminder to save it
/merit	Merit lookup	Member enters their 6-digit ID to see their merit points and the events they were checked in to
/checkin/:eventToken	Event check-in	Opened by scanning the live event QR. The page asks for location access, confirms the code, removes it from the address bar, then the member enters their 6-digit ID to be checked in
/leaderboard	Leaderboard	Optional ranking by points, showing name, rank and points (Proposed; see Appendix 4)

Build rules:

- Upcoming events, media items and page text come live from Supabase tables, never hard-coded.
- Photos are served from a public Supabase Storage bucket. Compress and lazy-load them.
- There is no member session. The three member actions are plain forms that call Edge Functions and show the result.
- Design mobile-first. Most members will reach /checkin and /register from a phone, often on limited data.

A.3 Member Flows

Flow	Steps	What the member sees
Register	Open /register, fill in the form and submit, then enter the verification code emailed to them	After verifying their email, a success page with their 6-digit membership ID, which is also emailed to them. If the email or phone is already registered, a clear message to contact the executives instead of a second ID
Look up merit	Open /merit, enter the 6-digit ID	First name, total points, and a list of events attended with the points each gave. An unknown ID gets a generic "ID not found"

Flow	Steps	What the member sees
		message
Check in to an event	Scan the live QR on the screen with the phone camera, which opens /checkin/:eventToken. The page asks to use the phone's location, confirms the code and moves to /checkin. Enter the 6-digit ID and submit within 3 minutes	"Checked in, +3 points" with their new total. Other outcomes: already checked in (no extra points), check-in closed, QR expired (scan the screen again), this phone has already checked someone in for this event, location off or outside the event area, ID not found

The ID is the only credential members hold, so the registration success page should tell them to save it (screenshot or copy).

A.4 Admin Section

The only part of the site with a login. Screens a user is not permitted to see are hidden from the menu (UX only; RLS is the real enforcement).

Screen	What it does
Overview	Total registered members, registrations this week, upcoming events, latest check-ins
Registrations	Table of every registered member: name, email, phone, programme, year, membership ID, registration date, total points. Search, filter and sort. **Export to CSV and Excel**
Events	Create, edit, publish and delete upcoming events, with a cover image (a published event appears on the Homepage and Events page straight away); set points per check-in (default 3); set the check-in window and the venue (tap "Use my current location" at the venue, then set the radius, 100 m by default). **Generate the event QR code**, shown full-screen on a projector as a live code that refreshes every 30 seconds (no download, because a saved image would defeat the rotation), and close check-in or reset the event's secret at any time
News	Write, edit, publish and delete news posts and announcements, with an optional cover image. Unpublish a post to hide it without deleting it
Attendance	Check-in list per event. Manual check-in by member ID for a member who cannot scan or whose location check failed. Remove a wrong check-in
Merit Tracker	Points history per member and total. Manual adjustment with a reason

Screen	What it does
	(Proposed)
Media & Content	Upload pictures (several at a time, optionally linked to an event) and delete them; add and remove YouTube links and recordings; manage the event calendar; edit homepage and About text
Messages	Read messages sent through the Contact Us form
Audit log	Read-only trail of admin actions, including exports, event secret resets and deletions

Deleting content. Every delete asks for confirmation. Deleting a picture also removes the file from Supabase Storage. An event that already has check-ins cannot be deleted, because that would remove members' merit points; the admin unpublishes it instead (Proposed). Resize large pictures in the browser before upload and accept only JPEG, PNG and WebP up to 5 MB .

A.5 Data You Will Read and Write (Supabase)

Public content is read with supabase-js and the anon key. Member actions go through Edge Functions. Admin screens read and write under RLS after the admin signs in.

Table / view	Who reads it, and how
site_content, events (published), news_posts (published), media_items (published)	Public pages, directly through RLS
members	Admin only (Registrations screen and export). Never readable by visitors
attendance, merit_records	Admin only. Members see their own through `merit_lookup`
leaderboard (view)	Public, if the committee approves it
contact_messages	Insert from the Contact Us form; admin read
audit_logs	Admin read-only (Technical Head)

Edge Functions to call (never build these; Backend owns them)

- `POST /functions/v1/register_member` — starts a registration and emails a verification code through Resend; then verify_email checks the code, creates the member and returns the 6-digit ID.
- `POST /functions/v1/merit_lookup` — takes a 6-digit ID and returns first name, total points and attendance list.
- `POST /functions/v1/check_in` — takes `{ session_id, member_code }` and records attendance and points. The session id comes from POST /functions/v1/start_check_in, which the check-in page calls once with the scanned QR token, the device token and the phone's location. check_in sends the location again.

- Admin actions that change data in a way that must be logged (reset an event secret, manual check-in, merit adjustment, role change, delete an event, news post or picture) also go through Edge Functions.

A.6 Role-Based UI

Only admins have accounts. The role comes from the user's JWT claim issued by Supabase Auth. Read it client-side to decide what to render. This is a UX convenience only; RLS is the real enforcement.

Role	Sees
Visitor / member (no account)	Public pages, registration, merit lookup, check-in
Communications Head	News, Media & Content, event publishing, contact messages (Proposed)
Organization Head	Events and QR generation, Attendance, Registrations and export, Merit Tracker (Proposed)
Technical Head	All admin screens, including role management and the audit log (Proposed)

The committee named these three heads as the only authorized admin users. The exact permission split above is *Proposed*, and so is who may export the registrations, because that file holds personal data. Both should be confirmed (Appendix 4).

A.7 Environment Variables (Frontend)

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
VITE_SITE_URL=            # base URL used inside event QR codes
VITE_CAPTCHA_SITE_KEY=    # Proposed, public key only
```

Only public values go here. Never place a service role key or any secret in the frontend.

A.8 Forms & Validation

Use React Hook Form + Zod for: registration (first name, last name, programme, year, email, phone number), the email verification code (6 digits), merit lookup and check-in (exactly 6 digits), event create/edit, media upload, manual adjustment, and the Contact Us form. Validate phone numbers and emails client-side, but treat server-side validation in the Edge Function as the real gate. Add a CAPTCHA or honeypot to the registration and Contact Us forms (Proposed).

A.10 Frontend Deliverables Checklist

Budget lines are to be agreed with the committee.

Deliverable	Item
Homepage and About page	1, 2
Design research note (SPE KNUST, SPE Stanford, others)	3
Registration page and ID success page	4
Merit lookup page and check-in page	5, 8
Admin shell with role-gated screens, Registrations table and export	6
Admin Events screen with event QR generation	8
News and Media sections, with admin managers for news, events and pictures	7
Merit Tracker admin screens	8

Part B — Backend & Infrastructure Development

Scope: the Supabase database, Row Level Security, Edge Functions, membership ID generation, event QR check-in and merit logic. This part holds every secret and every rule that must not be trusted to the client. Members have no accounts, so the three member actions (register, merit lookup, check-in) are public Edge Functions that must defend themselves against abuse.

B.1 Tech Stack

Layer	Choice
Database	PostgreSQL via Supabase
Auth	Supabase Auth for admins only, with Row Level Security (RLS)
Serverless functions	Supabase Edge Functions (Deno runtime)
File storage	Supabase Storage (news and event images, gallery photos), with upload and delete limited to admin roles
Bot protection	CAPTCHA such as Cloudflare Turnstile on registration (Proposed)
Email	Resend API to verify the email address and send the membership ID

B.2 Database Schema and RLS

Full table definitions are in Appendix 1. Backend owns the migrations, constraints, foreign keys and all RLS policies.

Principle. Visitors and members have no database access at all, except reading published public content. Everything they do goes through an Edge Function that runs with the service role key. The `members` table has RLS enabled and no public policy.

Admin roles. Do not use `auth.role()` for app roles. In Supabase it returns only `anon` or `authenticated`. Store admin roles in a `user_roles` table keyed to the Supabase Auth user, and check them with a helper function:

```
create type app_role as enum
('communications_head','organization_head','technical_head');

create function public.has_role(r app_role) returns boolean
language sql stable security definer set search_path = public as $$
  select exists (select 1 from user_roles
                 where user_id = auth.uid() and role = r);
$$;

alter table members enable row level security;

-- No policy for anon: visitors can never read members
create policy admin_read_members on members
  for select using (has_role('organization_head') or has_role('technical_head'));

-- Public may read only published events
create policy public_read_events on events
  for select using (is_published);
```

Write equivalent policies for every table below. Note that the public events policy must not expose `checkin_token`. Put the token in a separate admin-only table or use a view that omits it.

Table	Public	Admin heads
site_content	Read	Write: Communications Head, Technical Head
events (public columns), news_posts, media_items	Read published rows	Write per role
event_checkin_tokens	None	Organization Head, Technical Head
members	None	Read: Organization Head, Technical Head
attendance, merit_records	None	Read per role. Writes only through Edge Functions
contact_messages	Insert only	Read
audit_logs	None	Technical Head read

Edge Functions bypass RLS with the service role key.

B.3 Edge Functions to Build

Function	Caller	Responsibility
register_member	Public	Validate input, check CAPTCHA and rate limit, reject duplicate email or phone, store a pending registration and email a

Function	Caller	Responsibility
		verification code through Resend
verify_email	Public	Check the code (hash, expiry, attempt limit), create the member, generate the 6-digit ID, return it once and email it through Resend
merit_lookup	Public	Rate-limit, take a 6-digit ID, return first name, total points and attendance list only
check_in	Public	Validate the check-in session, check-in window, device, location and ID, record attendance and award points in one transaction
start_check_in	Public	Verify the signed QR token and its window and that the phone is inside the event's radius, then open a one-time check-in session (3 minutes) bound to the scanning device. The QR token is not needed after this step
qr_token, reset_event_secret	Admin	qr_token returns the current signed QR token for an event (the admin screen refreshes it every 30 seconds). reset_event_secret replaces the event's secret, which kills every outstanding QR and session; write audit log
manual_check_in, adjust_merit	Admin	Record attendance or a points adjustment with a reason, write audit log (Proposed)
send_membership_id	Called by verify_email	Email the member their ID once the address is verified
send_contact_notification	Database trigger	Notify executives of a new Contact Us message (Proposed)

B.4 Registration and the 6-Digit Membership ID

- Generate the ID on the server with a cryptographically secure random number between 100000 and 999999. Do not derive it from the name and do not count upward, so IDs cannot be guessed in order.

- Enforce uniqueness with a unique constraint on `members.member_code`. If an insert collides, generate a new number and retry.
- Reject a registration whose email or phone already exists (compare in a normalized form: lower-case email, phone in one format). Do not reveal the existing member's ID in the error message.
- Verify the email before issuing an ID. register_member stores a pending registration (it is not a member yet) and emails a 6-digit verification code with Resend. The code is stored hashed, expires after 10 minutes and allows 5 attempts. verify_email checks the code, creates the member, generates the ID and returns it once. The ID is also emailed so a lost ID can be recovered.
- Save the time the email was verified (`email_verified_at`). Registration collects personal data; see B.8.

B.5 Event QR Code and Check-In

The QR on the projector is live, only works at the venue, and ties each check-in to the phone that made it. A photo or a copied link stops working within a minute, a phone outside the venue range is refused, and one phone can check in only one member per event.

How check-in works

1. **Venue.** When an admin creates an event, they open it at the venue and tap "Use my current location". The backend saves the position and a radius (default 100 m) in `events`, and a random secret (at least 256 bits) in `event_checkin_tokens`. The secret never leaves the server.
2. **Rotating QR.** The admin's full-screen QR page asks `qr_token` for the current token every 30 seconds. A token is the event id, the current 30-second window number and an HMAC of both made with the event's secret. Nothing is written to the database when it rotates.
3. **Short life.** The QR encodes `https://<site>/checkin/<token>`. The server accepts a token only in its own window and the one after it, so a code is good for 60 seconds at most.
4. **Scan.** The page asks the phone for its location (browser Geolocation, which needs HTTPS) and calls `start_check_in` with the token, the device token and the position (latitude, longitude, accuracy). The server checks the signature, the window, the device and the location. If all pass, it opens a check-in session that works once and expires after 3 minutes. The session id stays in the page's memory and never appears in the address bar. The page then replaces the address with `/checkin`, so a link copied after scanning carries nothing useful.
5. **Enter ID.** The member enters their 6-digit ID. The page calls `check_in` with `{ session_id, member_code }` and the phone's position again. The QR token is not used again.
6. **Checks.** `check_in` checks, in order: the session exists, is unused, is unexpired and belongs to this device; the phone is still inside the venue range; the event is published and inside its check-in window; the ID belongs to a member; the member has not already checked in; this device has not already checked in someone for this event.
7. **Record.** In one transaction it marks the session used, inserts the `attendance` row with the device hash and rounded distance, and inserts a `merit_records` row for the event's points (default **3**). It returns the first name, points awarded and the new total.

Location rules

- The server measures the distance between the phone and the venue (Haversine formula). It accepts only if the phone's reported accuracy is 100 m or better and the distance is within the event's radius.
- Location is required. If the member denies permission, the phone cannot give a position, or the accuracy is too poor, check-in is refused with a clear message on how to allow location and scan again. An admin can then check the member in by hand from the Attendance screen.
- Coordinates are used in memory and never stored or logged. Only the result and the distance, rounded to 10 m, are saved in `attendance.distance_m`. The check-in page shows a one-line notice that location is used only to confirm the member is at the event.
- Set the radius to cover the room with a margin for indoor GPS error. Too small a radius refuses real members. Admins can change it before the event, and should test it inside the actual venue.

One phone, one check-in per event

- On its first visit the page creates a random device token and keeps it in the browser's local storage. The backend stores only a hash of it (with a server-side pepper) in `attendance.device_hash`.
- `attendance` has unique constraints on `(event_id, member_id)` and on `(event_id, device_hash)`. The second person to try on the same phone is told the phone has already been used for this event. Points are never awarded twice.
- Optional setting, off by default (Proposed): lock a phone to one member across all events. The first member ID a device checks in with is bound to it in a `devices` table, and later events from that device must use the same ID. An admin can reset the binding when a member changes phone.

Other rules

- The check-in window defaults to the event's start and end time, with a short grace period (Proposed). Outside it, the function refuses.
- An admin can close check-in at any time, or reset the event's secret, which kills every outstanding QR and session at once.
- Rate-limit per IP, per device and per ID, and return the same generic message for a wrong ID so IDs cannot be tested one by one. Do not block by IP alone, because many members share the campus Wi-Fi.
- The admin Attendance screen flags bursts of check-ins from the same IP and browser type for review (Proposed).

B.6 Merit Tracker Logic

- `merit_records` is a ledger. Each row is one change to a member's points, with the member, the event (if any), the points, the source (`event_checkin` or `manual_adjustment`), the reason and who made it.
- A check-in adds a row of +3 points (the event's `points` value, default 3). The member's total is the sum of their rows.
- Manual adjustments are admin-only, need a reason, and are logged (Proposed).
- `merit_lookup` returns the total and the events attended. It returns no email, phone or other personal data.

- The `leaderboard` view ranks members by total points and exposes only a display name, rank and points. Publish it only if the committee approves.

B.7 Automated Processes

Process	Frequency
Database backup	Daily
Leaderboard refresh (only if the view is materialized)	Hourly (Proposed)
Cleanup of expired rate-limit records and check-in sessions	Daily (Proposed)

Check-in windows need no scheduled job. The `check_in` function compares the current time to the event's window on every call.

B.8 Security Implementation

Authentication & RBAC

- Supabase Auth is used for admins only. App roles live in `user_roles` and are checked through `has_role()`. Tokens expire hourly with refresh-token rotation.
- Admin roles (Communications Head, Organization Head, Technical Head) are assigned only by the Technical Head. Every role change is written to `audit_logs`.

Public endpoint protection (register_member, merit_lookup, check_in, Contact Us)

- Rate-limit per IP, and for `merit_lookup` and `check_in` also per member ID. A 6-digit ID has only 900,000 possible values, so unlimited guessing must not be possible.
- Return generic errors that do not say whether an ID exists.
- `merit_lookup` returns only first name, points and attendance, never contact details.
- CAPTCHA on registration and Contact Us (Proposed).
- Validate every input with Zod and use parameterized queries only.
- CORS restricted to the approved site origin.

Data security

- Registrations hold personal data (name, email, phone, programme). RLS blocks all public access. Only the authorized heads can read or export it.
- Log every registrations export in `audit_logs` with who and when.
- Ghana's Data Protection Act, 2012 (Act 843) applies to this personal data. Show a short data-protection notice under the registration form (Proposed) and on the check-in page, where the phone's location is requested. Location is checked and discarded; only the result and a rounded distance are kept. The committee should confirm its data-handling obligations (Appendix 4).
- Public Storage bucket holds only photos meant to be public.
- TLS everywhere; automated daily backups.

B.9 Environment Variables / Secrets (Backend only)

```
SUPABASE_SERVICE_ROLE_KEY=
CAPTCHA_SECRET_KEY=       # Proposed
```

```
RESEND_API_KEY=
DEVICE_HASH_PEPPER=
```
These live only in Supabase Edge Function secrets. Never in frontend code, a client bundle, or a public repository.

B.10 CI/CD

```
name: Deploy to Production
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
      - run: npm run build
      - name: Deploy to Vercel
        run: vercel --prod --token=$VERCEL_TOKEN
      - name: Deploy Edge Functions
        run: supabase functions deploy --project-ref=$SUPABASE_PROJECT_REF
```
Run database migrations through the Supabase CLI. Use a separate staging project for testing.

Appendix 1 — Shared Database Schema Reference

Authoritative for both Frontend (read/query shape) and Backend (constraints, migrations, RLS). All tables carry `created_at` and `updated_at` unless noted. Table names are *Proposed*; the committee's requirements define the features, not the tables.

```
members                                -- no auth account; created by register_member
  id (UUID, PK)
  first_name, last_name
  email (text, unique, normalized)
  phone (text, unique, normalized)
  programme, year_of_study
  member_code (char(6), unique)        -- 6-digit membership ID, random, 100000-999999
  email_verified_at (timestamp)

email_verifications                    -- pending registrations, removed after use or
expiry
  id (UUID, PK)
  email (text)
  registration_data (jsonb)            -- first_name, last_name, programme, year,
phone
  code_hash (text)                     -- hash of the 6-digit verification code
  expires_at (timestamp)
  attempts (integer, default 0)

user_roles                             -- admins only
  user_id (UUID, FK -> auth.users.id)
  role (ENUM: communications_head, organization_head, technical_head)
  PK (user_id, role)
```

```
site_content
  key (text, PK)                      -- e.g. home_intro, about_mission, about_vision
  body (text)

events
  id (UUID, PK)
  title, slug (unique), description
  starts_at, ends_at (timestamp)
  location (text)
  cover_image_url (text)
  points (integer, default 3)         -- points awarded per check-in
  checkin_opens_at, checkin_closes_at (timestamp)
  venue_lat, venue_lng (numeric)      -- captured by the admin at the venue
  radius_m (integer, default 100)     -- allowed distance from the venue
  is_published (boolean)

event_checkin_tokens                  -- admin-only, never public
  event_id (UUID, FK -> events.id)
  secret (text)                -- random, 256-bit; signs the rotating QR tokens,
never leaves the server
  is_active (boolean)
  rotation_seconds (integer, default 30)    -- how often the QR token changes

attendance
  id (UUID, PK)
  event_id (UUID, FK -> events.id)
  member_id (UUID, FK -> members.id)
  checked_in_at (timestamp)
  method (ENUM: qr, manual)
  device_hash (text, nullable)        -- hash of the device token; null for manual
check-ins
  distance_m (integer, nullable)      -- distance from the venue, rounded to 10 m; no
coordinates stored
  UNIQUE (event_id, member_id)
  UNIQUE (event_id, device_hash)      -- one device, one check-in per event

merit_records                         -- points ledger
  id (UUID, PK)
  member_id (UUID, FK -> members.id)
  event_id (UUID, FK -> events.id, nullable)
  points (integer)
  source (ENUM: event_checkin, manual_adjustment)
  reason (text, nullable)
  awarded_by (UUID, FK -> auth.users.id, nullable)

leaderboard (view)                    -- optional
  member_id, display_name, total_points, rank

news_posts
  id (UUID, PK)
  title, slug (unique), body (text)
  cover_image_url (text, nullable)
  is_published (boolean)
  published_at (timestamp)
```

```
  created_by (UUID, FK -> auth.users.id)

media_items
  id (UUID, PK)
  type (ENUM: youtube, recording, photo)
  storage_path (text, nullable)       -- uploaded file; deleted from Storage with the
row
  title, url (text)
  event_id (UUID, FK -> events.id, nullable)
  is_published (boolean)

contact_messages
  id (UUID, PK)
  name, email, message (text)
  is_read (boolean)

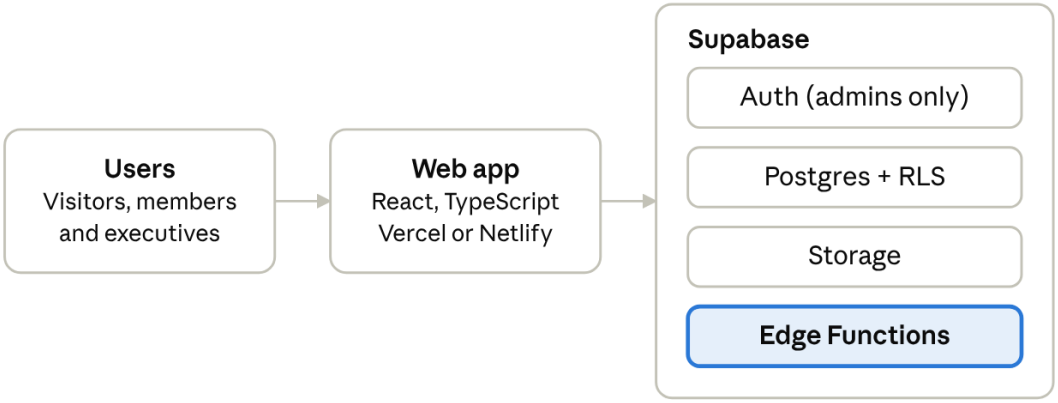
audit_logs
  id (UUID, PK)
  user_id (UUID, FK -> auth.users.id, nullable)
  action (text)                       -- e.g. registrations_exported,
event_secret_reset
  details (jsonb)
  ip_address (inet)
  created_at (timestamp)

checkin_sessions                      -- one-time sessions opened by scanning the QR
  id (UUID, PK)
  event_id (UUID, FK -> events.id)
  device_hash (text)
  expires_at (timestamp)              -- 3 minutes after the scan
  used_at (timestamp, nullable)

devices                               -- Proposed: device lock across events
  device_hash (text, PK)
  member_id (UUID, FK -> members.id)
  first_seen_at (timestamp)
```

Appendix 2 — High-Level Architecture

Every request goes through the web app into Supabase



Appendix 3 — Key Risks to Flag Early

Risk	Mitigation
Check-in without login: a member's ID can be used by someone else, or a QR photo or link shared with people not at the event (someone with a location-spoofing app, a friend inside the room using a member's ID, or someone clearing browser data to get a new device token can still get through)	Rotating signed QR (30 seconds); one-time check-in session tied to the scanning phone; one check-in per phone per event; location check against the venue radius; check-in window; rate limits; admin can review and remove check-ins; reset the event secret instantly
Guessing 6-digit IDs to view other members' points or check them in	Random IDs; rate limit per IP and per ID; generic errors; lookup returns only first name and points
Members lose their ID	Email a copy at registration; admins can look an ID up
Duplicate or fake registrations	Unique email and phone; CAPTCHA; rate limit
Personal data exposed or over-shared	RLS on every table; no public reads; export limited to named roles and logged
QR will not scan or page is slow on phones	Show the check-in link in text under the QR; mobile-first page; test on low-end phones and slow networks
Content not ready at launch	Collect photos and text early; keep content editable by admins after launch

Risk	Mitigation
Executive handover (heads change each year)	Roles assigned in `user_roles`, not hard-coded; documented handover steps
Real members refused because GPS is inaccurate indoors or location is switched off	Radius with a margin (100 m default); clear on-screen message; admin manual check-in; test inside the real venues before launch